

残差ブロックの導入による U-Net の高度化とセマンティックセグメンテーションの精度向上

2026 年 6 月 17 日

四国大学文学部書道文化学科 3 年 岡野 菜月

1. 要旨

本実装では、U-Net 全体の構造を大きく変更するのではなく、Encoder 部分のみを Residual Block に置き換えた。U-Net は Encoder-Decoder 型のモデルであり、Encoder で画像の特徴抽出を行い、Decoder で空間解像度を回復しながらセグメンテーションマップを生成する。また、Skip Connection によって Encoder の浅い層の情報を Decoder に直接伝えることで、空間的な情報を補完できる。

そこで本実装では、U-Net の基本構造である Decoder および Skip Connection は維持したまま、Encoder の各層を Residual Block で構成した。Residual Block を積み重ねることで、層をまたいで入力情報を深い層へ伝えやすくなり、特徴の保持や学習の安定化が期待できる。このようにして、U-Net の空間情報を活用する利点を保ちながら、Encoder における特徴抽出能力の向上を試みた。

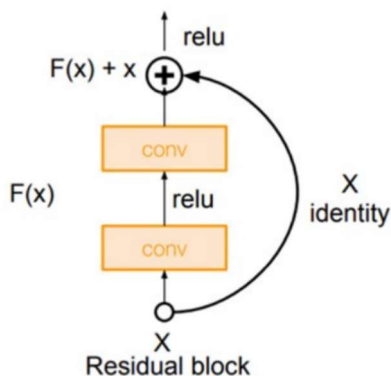
2. モデル構成

2.1. Residual Block の定義

Residual Block では、主経路において 2 層の畳み込みによって入力特徴を変換する。1 層目では、 3×3 の畳み込み、バッチ正規化、ReLU を適用し、2 層目では再び 3×3 の畳み込みとバッチ正規化を適用する。その後、これらの処理によって得られた特徴 $f(x)$ に対して、Residual Block への入力 x をショートカット経路から加算し、最後に ReLU を適用して出力を得る。

ショートカット経路では、入力と出力のチャンネル数が異なる場合に 1×1 の畳み込みを用いて次元を一致させる。一方でチャンネル数が同じ場合には入力をそのまま通すことで残差接続を実現する。

図 1. Residual Block の模式図



"Keep Your AI-es on the Road: Tackling Distracted Driver Detection with Convolutional Neural Networks and Targeted Data Augmentation" より引用

コード 1. Residual Block の実装

```
# 残差ブロック (ResidualBlock) の定義
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        # 主経路: 2層の畳み込みで入力特徴を変換
        self.conv = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_channels)
        )
        self.relu = nn.ReLU(inplace=True)

        # ショートカット経路: 入出力のチャンネル数が異なる場合は、1x1畳み込みでチャンネル数を合わせる
        if in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, kernel_size=1, bias=False),
                nn.BatchNorm2d(out_channels)
            )
        else:
            # チャンネル数が同じ場合は入力をそのまま通す
            self.shortcut = nn.Identity()

    def forward(self, x):
        # 主経路: 畳み込みで入力 x を変換し、特徴を抽出
        out = self.conv(x)
        # ショートカット経路の出力を加算して残差接続を実現
        out += self.shortcut(x)
        # 活性化関数 ReLU を適用して非線形性を導入
        return self.relu(out)
```

2.2. U-Net の Encoder 部分

U-Net の Encoder 部分では、Residual Block を積み重ねることで、入力画像から段階的に特徴を抽出する構成とした。各層の後には 2×2 の最大プーリングを適用し、特徴マップをダウンサンプリングすることで空間サイズを縮小した。これにより、浅い層では局所的な特徴を捉え、より深い層では抽象的な特徴表現を獲得できるようにした。

コード 2. U-Net の Encoder 部分の実装

```
class UNet(nn.Module):
    def __init__(self, in_channels, num_classes):
        super().__init__()
        # エンコーダ部分: Residual Blockを重ねて特徴を抽出
        self.enc1 = ResidualBlock(in_channels, 64)
        self.enc2 = ResidualBlock(64, 128)
        self.enc3 = ResidualBlock(128, 256)
        self.enc4 = ResidualBlock(256, 512)
        # 最大プーリング: 各フィルタの最大値を取り、特徴マップをダウンサンプリング
        self.pool = nn.MaxPool2d(2)
```

3. 性能評価

本実装の性能を評価するため、検証用データに対して mean Intersection over Union (mIoU) を算出した。また、予測結果の傾向を視覚的に確認するため、Segmentation Map の可視化も行った。

3.1. mIoU の測定結果

検証用データに対して算出した mIoU は 0.8956 であり、提出後にテストデータの正解ラベルに対して測定された mIoU は 0.3991 であった。両者には大きな差が見られ、この差は評価対象データの違いや評価条件の差異によって生じた可能性がある。

評価用データに対する mIoU は、各クラスについて TP、FP、FN を集計し、IoU を算出した後にその平均を取ることで求めた。以下に、mIoU の評価に用いた関数を示す。

コード 3. mIoU の評価関数

```
def evaluate_model(model, loader, num_classes=13, ignore_index=255):
    model.eval()

    total_tp = np.zeros(num_classes, dtype=np.float64)
    total_fp = np.zeros(num_classes, dtype=np.float64)
    total_fn = np.zeros(num_classes, dtype=np.float64)

    with torch.no_grad():
        for image, depth, label in loader:
            image, depth = image.to(config.device), depth.to(config.device)
            x = torch.cat((image, depth), dim=1)

            # 推論
            output = model(x)
            pred = output.argmax(dim=1).cpu().numpy()
            gt = label.cpu().numpy()

            mask = (gt != ignore_index)

            for cls in range(num_classes):
                total_tp[cls] += np.sum((pred == cls) & (gt == cls) & mask)
                total_fp[cls] += np.sum((pred == cls) & (gt != cls) & mask)
                total_fn[cls] += np.sum((pred != cls) & (gt == cls) & mask)

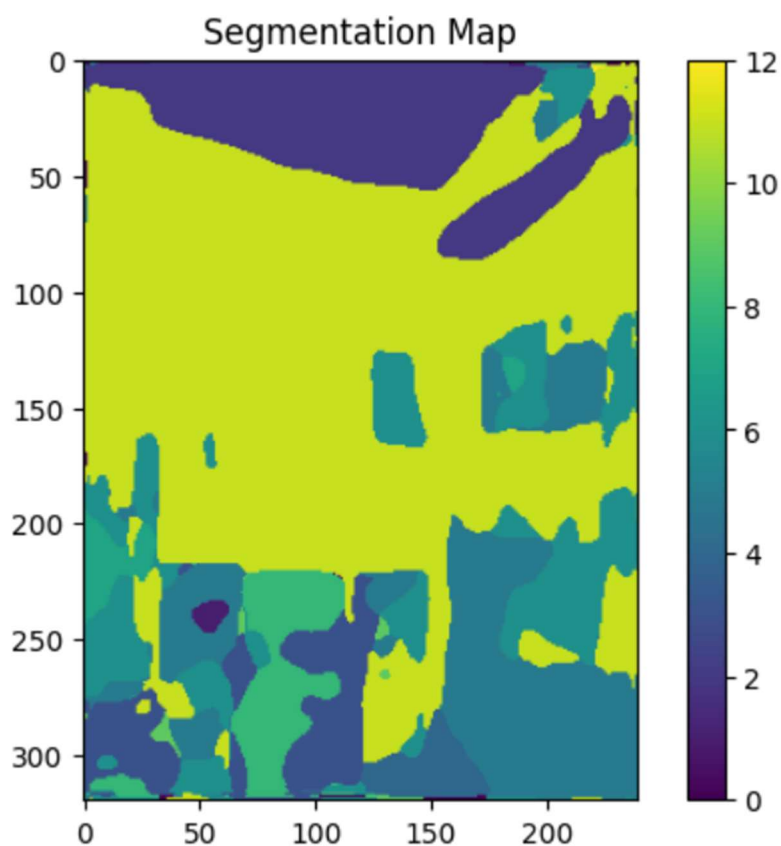
    # mIoU 計算
    ious = []
    for cls in range(num_classes):
        denominator = total_tp[cls] + total_fp[cls] + total_fn[cls]
        if denominator == 0:
            ious.append(np.nan)
        else:
            ious.append(total_tp[cls] / denominator)

    return np.nanmean(ious)
```

3.2. セグメンテーションマップの可視化

モデルの予測傾向を定性的に確認するため、セグメンテーションマップの可視化を行った。図2に予測結果の一例を示す。可視化結果では、クラス0から12までが色分けされた凡例によって識別可能であり、各クラスの予測領域を視覚的に確認することができた。また、画像全体の大まかな領域構造に加え、細かな領域についても一定程度表現できていることが分かった。このことから、本モデルは広域の特徴だけでなく、細部の空間情報についても一定程度捉えられていると考えられる。

図2. セグメンテーションマップの可視化結果。クラス0から12までを色分けして表示している。



4. 参考文献

[Mofid+ 2006] “[2006.10955] Keep Your AI-es on the road: Convolutional Neural Networks とターゲットデータ拡張による注意散漫ドライバー検出への取り組み” [arXiv:2006.10955](https://arxiv.org/abs/2006.10955)